Introduction

As you go through each phase of the term project, it's important to keep in mind the information presented in the following sections:

- overview
- description of the project structure

Phase 4 of the term project is worth 20% of your overall project grade. It is not necessary for you to pass this phase of the term project to pass the course. However, you MUST earn an overall passing grade for the term project. So, if you have not done well on previous phases of the term project, it may be necessary for you to do well on this one.

Academic Honesty

You are reminded that all the code that you submit at each phase of the term project must be

- code that you have written yourself since the start of this term (you must not make use of any code that you have written in the past)
- code found in or based on any of the projects that we present to you in lecture during the term, provided you insert a comment into your code to attribute the source

Please ensure that you do not provide other students with access to your code. By doing so, you risk enabling academic misconduct for which you will be held accountable. Typically, the same grade penalty is applied to the student who enabled access to their code, as to the student who used the code.

If you find the project challenging, ask for help from a course instructor or from one of the TAs *assigned to the course this term*. Do not be tempted to copy code from someone else and do not, under any circumstances, allow another student access to your code in any way.

General Requirements

You should commit and push your code to GitHub after completion of each task, if not more frequently. By pushing your code to GitHub you are effectively making a backup of your work which you'll be able to retrieve if anything happens to the code on your laptop.

All code must be appropriately documented:

- each class should have a class-level comment (that appears just before the class declaration) to describe the information that the class represents
- every method must be documented with requires, modifies and effects clauses, as appropriate (note that simple setter and getter methods, and test methods, do not have to be documented)

While marks are not awarded for documentation, penalties of up to 10% will be applied at each phase of the term project for missing documentation.

Main Tasks

In Phase 4 you will continue to work with the same repository that was provided to you for Phase 1.

Task 1

If you have not done so already, please ensure that you have committed your Phase 3 code, pushed it to GitHub and requested a grade from AutoTest for your Phase 3 deliverables. We then have a record of the work that you completed during Phase 3, so that you may start work on Phase 4.

Task 2

It's useful for debugging to log events in an application. In the context of your project, we may want to log when an X is added to Y, removed from Y, when Y is cleared, or whatever interesting actions occur on the Xs and Y.

In this task, you add event logging to your project. You **must** re-use the `Event` and `EventLog` classes from the Alarm System application for your logging. Add these classes to your `model` package and do **not** modify them in any way, other than to fix checkstyle issues. Understand their use in the Alarm System before using them yourself. In particular, note that the `EventLog` constructor is private and cannot be called outside that class. So, your application must use the `getInstance` method. (This is an application of the Singleton design pattern! The pattern is appropriate here, as we want all events to be added to the same, shared log object, and we want the event log accessible from all parts of the application.) You are also welcome to copy and adapt the test classes for `Event` and `EventLog`.

For this task you must:

- make calls to the `logEvent` method of the `EventLog` class from code in your `model` package **but from nowhere else**. In particular, **you must not call this method from anywhere in your `ui` package**.
- In Phase 3 you displayed all Xs in your Y and supported two actions related to the Xs and Y. You **must** log the key events in your model related to those actions. (You may log other actions in your model if you wish.) You may **NOT** log anything directly in the `ui` package. For example, if one action in your GUI was to add an X to a Y, you must log the key event(s) that occur during that action in your model. Be sure to use a description that accurately conveys the event that has occurred - so "appointment added to calendar" rather than "an event happened!". Since you are not logging in the UI, none of these will be UI events (like buttons clicked or scrollbars moved) but rather model changes (like an X added or an important field updated). (If nothing changes in your model in response to key actions in your UI, something is wrong!)
- when the user quits your application, print *to the console* all the events that have been logged since the application started - you **must not** print the messages to console as the events occur. You may find Oracle's tutorial on Window events helpful.
- copy and paste a representative sample of the events that occur when your program runs and add it to a section entitled "Phase 4: Task 2" at the end of your README.md file. If it is helpful to do so, you may wish to refer to the UML class diagram that we ask you to draw in Task 3.

Note that `EventLog` implements the `Iterable` interface. We'll be looking into this in more detail during our last module at the end of term. For now, be aware that this allows client code to use a for-each loop to iterate over the events in an `EventLog` object. See the `printLog` method of the `ScreenPrinter` class in the Alarm System application for an example of how this is done.

Commit and push your work to GitHub using the commit comment "Phase 4: Task 2"

Task 3

Draw a UML class design diagram that represents the final version of your code.

Your diagram must include:

- all classes/interfaces in your application **except** exceptions and test classes
- all subtyping relationships (extends/implements)
- associations between elements in the diagram (but not with other classes like your own exceptions or built-in classes), **with** multiplicities
- class names and <<abstract>>/<<interface>> decorations where appropriate

You should omit:

- dependency arrows
- all other classes (exceptions, test classes, built-in Java classes, third-party libraries, etc.)

- methods/fields in your class/interface boxes (so just a type name and maybe an abstract/interface decoration in each box)

You can draw your UML class design diagram using any tool that you wish, you may also draw it neatly by hand. When you are done, export your UML class diagram as a PDF, JPG or PNG file. If you drew your diagram by hand, take a picture of it and save it as a PDF, JPG or PNG file. **Name the file UML_Design_Diagram** with an extension appropriate for the type of file (e.g. UML_Design_Diagram.pdf) and **add it to the root of your project in IntelliJ**. You can just drag it and drop it onto the root folder. **Be sure to add the file to version control** (right-click and select Git → Add...), and then commit and push your work to GitHub.

YOU ARE RESPONSIBLE for viewing your project on GitHub to ensure the diagram is correct there! Files missing on GitHub receive no marks.

Add a section at the end of your README.md file entitled "Phase 4: Task 3". Reflect on the design presented in your UML class diagram. If you had more time to work on the project, what substantive refactoring might you use to improve your design? In 1-2 paragraphs in this section of your README, **describe the refactoring you are imagining and your reasons for it**. You **do NOT need** to perform the refactoring - just identify the changes you would make if you had more time. Keep in mind that refactoring does **not** mean adding more features to your application.

(Everyone can find some refactoring. If you believe there's **no** refactoring that could improve your design: Check again; there is! Even if you still think there's nothing, there **are** tradeoffs you made that could be made differently via a refactoring. Choose one, follow the description above, but emphasize the advantages and disadvantages of the proposed refactoring.)

Grading

When you are ready, commit and push your code to GitHub. Request feedback from AutoTest by making the following comment on your commit:

```
@210-bot #project
```

As you are all working on individual projects, AutoTest will obviously not give feedback on the correctness of your code. However, it will check that your code adheres to the CPSC 210 coding style. In Phase 4, AutoTest is being used only to verify that your code passes the checkstyle test. The code coverage and test scores will not factor into your grade for Phase 4.

Your grade for Phase 4 will be determined as follows:

- 40 pts - completion of Task 2, including the addition to the README.md file. (If no events are logged when your application runs, partial credit is available for clearly explaining in your README what went wrong as long as you appropriately attempted to log events in your model.)
- 60 pts - completion of Task 3, including the addition to the README.md file
- a **penalty** of 20 pts will be applied to any submission that fails checkstyle
- a **penalty** of 20 pts will be applied if we are able to view your UML diagram in your github repository but it is not in the expected location or file format. (If we cannot find or view your file in your github repository, all points associated with the diagram will be deducted. Be sure **you** double-check on github.students.cs.ubc.ca that your file is present, in the expected location, and in one of the expected file formats.)

< Previous Next >

© All Rights Reserved